

Ex:No: 2

Date: 19/2/26

Data Preprocessing and Integration of Customer Purchase Data from Excel and Relational Database

Problem Description:

A retail organization maintains customer purchase information for business analytics. The data is stored across **two different sources**:

- **Online purchases** → Excel workbook
- **In-store purchases** → Relational database (SQLite)

Each dataset contains the following attributes:

- **CustomerID** – Unique identifier for each customer
- **Age** – Numerical attribute representing customer age
- **PurchaseAmount** – Numerical attribute representing purchase value
- **Membership** – Categorical attribute indicating membership type (Gold, Silver, Regular)
- **City** – Categorical attribute representing customer location

Due to manual data entry and synchronization issues, the datasets contain **missing values**, **duplicate records**, and **inconsistent formats**.

The organization needs to load both datasets into R, integrate them, explore the data, separate numerical and categorical attributes, clean the data, and prepare a consistent dataset for analysis.

Objective:

To load customer purchase data from an **Excel workbook** and a **relational database** into R, integrate the datasets, explore the data, separate numerical and categorical attributes, handle missing values and duplicates, convert categorical variables into factors, and prepare a clean dataset for business analysis.

Data Preprocessing

Data preprocessing in R is a crucial step to prepare your data for analysis or machine learning tasks. Below are some common steps along with example code snippets.

Steps in Data Preprocessing

1. Data Collection:

- Load data from various sources such as CSV files, Excel files, databases, etc.

2. Data Cleaning:

- Handle missing values. (In R, the `na.rm` parameter is used to exclude missing values (NA) during calculations. This ensures accurate results by ignoring NA values.)
- Remove duplicates.
- Correct errors and inconsistencies.

3. Data Transformation:

- Normalize or standardize data.
- Encode categorical variables.
- Apply feature scaling techniques.

4. Data Integration:

- Combine data from multiple sources.
- Resolve schema and data conflicts.

5. Data Reduction:

- Reduce the size of the dataset while retaining important information.
- Use techniques like Principal Component Analysis (PCA) or feature selection.

6. Data Splitting:

- Split the data into training and testing sets.

Data Preprocessing Tasks

- Data collection from relational database
- Data integration from multiple tables
- Handling missing numerical and categorical values
- Removing duplicate records
- Converting categorical variables into factors
- Preparing a clean dataset for analysis

Procedure:

- Step 1: Load Required Packages

`install.packages("openxlsx")`

`install.packages("DBI")`

`install.packages("RSQLite")`

`library(openxlsx)`

`library(DBI)`

`library(RSQLite)`

→ print (online-data)

CustomerID	Age	PurchaseAmount	Membership	City
1	25	1800	Gold	Chennai
2	30	1500	Silver	Bangalore
3	NA	1800	Gold	Chennai
4	45	NA	NA >	Mumbai
5	40	2000	Silver	NA >

> print (ingstore-data)

CustomerID	Age	PurchaseAmount	Membership	City
1	35	1600	Gold	Chennai
2	45	1700	Reguler	Mumbai
3	28	1400	Silver	Delhi
4	NA	1300	Gold	Delhi
5	NA	1300	Gold	Delhi

• Step 2: Create and Store Excel Dataset (Online Purchases)

CustomerID	Age	PurchaseAmount	Membership	City
1	25	1200	gold	Chennai
2	30	1500	silver	Bangalore
3	NA	1800	gold	Chennai
4	45	NA	<NA>	Mumbai
5	40	2000	silver	<NA>

Online-data ← data_frame

CustomerID = c(1,2,3,4,5),

Age = c(25,30, NA,45,40),

~~PurchaseAmount~~ = c("Gold", "Silver", "Gold", NA, "Silver"),

~~Membership~~ = c(*),

PurchaseAmount = c(1200, 1500, 1800, NA, 2000),

City = c("Chennai", "Bangalore", "Mumbai", "Mumbai", "Mumbai")

#createWorkbook wb

wb ← createWorkbook()

add_worksheet(wb, "Online Purchases")

writeData(wb, "OnlinePurchases", online-data)

saveWorkbook(wb, "online-purchases.xlsx", overwrite="true")

= Print output online_data

Print (online-data)

- Step 3: Create and Store SQLite Dataset (In-Store Purchases)

	CustomerID	Age	PurchaseAmount	Membership	City
1	3	35	1600	Gold	Chennai
2	4	45	1700	Regular	Mumbai
3	6	28	1400	Silver	Delhi
4	7	NA	1300	Gold	Delhi
5	7	NA	1300	Gold	Delhi

```

con <- dbConnect(SQLite(), "store-purchases.db")
instore_data <- data.frame(
  CustomerID = c(3, 4, 6, 7, 7),
  Age = c(35, 45, 28, NA, NA),
  PurchaseAmount = c(1600, 1700, 1400, 1300, 1300),
  Membership = c("Gold", "Regular", "Silver", "Gold", "Gold"),
  City = c("Chennai", "Mumbai", "Delhi", "Delhi", "Delhi")
)
dbWriteTable(con, "InstorePurchase", instore_data, overwrite=TRUE)
dbDisconnect(con)
# Print output instore_data

Print(instore_data).

```

- Step 4: Load Both Datasets into R

```

excel_df <- read.xlsx("Online-purchase.xlsx")
con <- dbConnect(SQLite(), "store-purchases.db")
db_df <- dbReadTable(con, "InstorePurchase")
dbDisconnect(con)

```

→ Print (customer_data)

customer ID	Age	Purchase Amount	Membership	City
1	25	1200	Gold	Chennai
2	30	1500	Silver	Bangalore
3	NA	1800	Gold	Chennai
4	45	NA	NA	Mumbai
5	40	2000	Silver	NA
6	35	1600	Gold	Chennai
7	45	1700	Regular	Mumbai
8	28	1400	Silver	Delhi
9	NA	1800	Gold	Delhi
10	NA	1300	Gold	Delhi

→ str (customer_data)

data frame: 10 obs. of 5 variables:

\$ customerID : num 1 2 3 4 5 6 7

\$ age : num 25 30 NA 45 40 35 45 28 NA NA

\$ Purchase Amount : num 1200 1500 1800 NA 2000 1600 1700 1400 NA 1800

\$ Membership : chr "Gold" "Silver" "Gold" NA...

\$ City : chr "Chennai" "Bangalore" "Chennai" "Mumbai"...

→ Summary (customer_data)

```

customerID      age      Purchase Amount      Membership      City
Min.   : 1.00   Min.: 25.00   Min.: 1200   Min.: Gold   Min.: Chennai
1st Qu.: 3.00   1st Qu.: 30.00   1st Qu.: 1500   1st Qu.: Gold   1st Qu.: Bangalore
Median: 4.00   Median: 36.00   Median: 1500   Median: Gold   Median: Chennai
Mean   : 4.20   Mean: 35.43   Mean: 1533   Mean: Gold   Mean: Mumbai
3rd Qu.: 5.75   3rd Qu.: 40.00   3rd Qu.: 1700   3rd Qu.: Gold   3rd Qu.: Delhi
Max.   : 7.00   Max.: 45.00   Max.: 2000   Max.: Gold   Max.: Delhi
NA's   : 9      NA's : 1
  
```

• Step 5: Integrate Datasets

customer_data ← rbind (credit_df, db_df)

Print integrated dataset customer_data

Print (customer_data)

• Step 6: Data Exploration

Structure customer_data. Use str()

str (customer_data)

Summary customer_data

Summary (customer_data)

Sum of Missing values based on column in customer_data

Sum (duplicated (customer_data))

7. `is.na(customer_data)`

Customer ID	Age	Purchase Amount	Membership	City
0	3	1	1	1

7. `sum(duplicated(customer_data))`

1

7. `table(customer_data$membership)`

Label	Regular	silver
5	1	3

7. `table(customer_data$city)`

Bangalore	Chennai	Delli	Mumbai
1	3	3	2

7. `print(numerical_data)`

	CustomerID	Age	Purchase Amount
1	1	25	1800
2	2	30	1500
3	3	NA	1800
4	4	45	NA
5	5	40	2000
6	3	35	1600
7	4	45	1700
8	6	28	1400
9	7	NA	1300
10	7	NA	1300

Sum of Duplicate records in customer_data

`sum(duplicated(customer_data))`

Categorical distribution Membership and City in customer_data. Use table()

`table(customer_data$membership)`

`table(customer_data$city)`

• Step 7: Separate Numerical and Categorical Data

`numerical_data <- customer_data[, sample(customer_data, 5, replace=TRUE)]`

`categorical_data <- customer_data[, sample(customer_data, 5, replace=TRUE)]`

Print numerical data

`print(numerical_data)`

Print categorical data

`print(categorical_data)`

Print (categorical - data)

	Membership	City
1	Gold	Chennai
2	Silver	Bangalore
3	Gold	Chennai
4	2NR	Mumbai
5	Silver	2NR
6	Gold	Chennai
7	Regular	Mumbai
8	Silver	Dallas
9	Gold	Dallas
10	Gold	Dallas

• Step 8: Data Cleaning

Handle missing numerical values in Age and PurchaseAmount

numerical_data['age'].na (numerical_data['age'])

mean (numerical_data['age', na.rm = TRUE])

numerical_data['PurchaseAmount'].na (numerical_data['PurchaseAmount'])

mean (numerical_data['PurchaseAmount', na.rm = TRUE])

mean (numerical_data['PurchaseAmount', na.rm = TRUE])

Handle missing categorical values Membership and City

categorical_data['membership'].na (categorical_data['membership'])

if membership[0] == "Regular"

categorical_data['city'].na (categorical_data['city'])

Remove duplicates

categorical

final_data <- cbind (numerical_data, categorical_data)

final_data <- final_data[! duplicated (final_data),]

Convert categorical to factor Membership and City

final_data\$membership <- factor (final_data\$membership)

final_data\$city <- factor (final_data\$city)

Print (final data)

	id	name	age	purchase amount	membership	city
1	1	Urfenai	25.00000	1200.000	Gold	Chennai
2	2	8	30.00000	1500.000	Silver	Bangalore
3	3		45.42857	1800.000	Gold	Chennai
4	4		45.00000	1533.333	Regular	Mumbai
5	5		40.00000	800.000	Silver	Unknown
6	6	3	35.00000	1600.000	Gold	Chennai
7	7	4	45.00000	1700.000	Regular	Mumbai
8	8	6	38.00000	1400.000	Silver	Delhi
9	9	7	35.42857	1300.000	Gold	Delhi

Print final cleaned dataset

Print (final-data).

Result:

Thus, the program successfully demonstrates importing data from an Excel workbook and a relational database, integrating multiple sources into a single dataset, exploring the data structure and identifying missing values and duplicates, separating numerical and categorical data, cleaning missing values and duplicates, converting categorical variables into factors, and producing a clean dataset ready for accurate business analysis.